作品紹介

作品名　：The Knight of FUGEDA
ジャンル：3Dアクション
開発環境：DXライブラリ、Visual Studio 2022
言語　　：C++
制作期間：2か月
制作人数：1人
担当箇所：プログラム全般
動作環境：Windows(XInputコントローラー対応)
GitHub　：https://github.com/KaminagaAyumu/TheKnightOfFugeda.git
動画　　：https://youtu.be/7CQtxcG7fe4



ゲーム概要

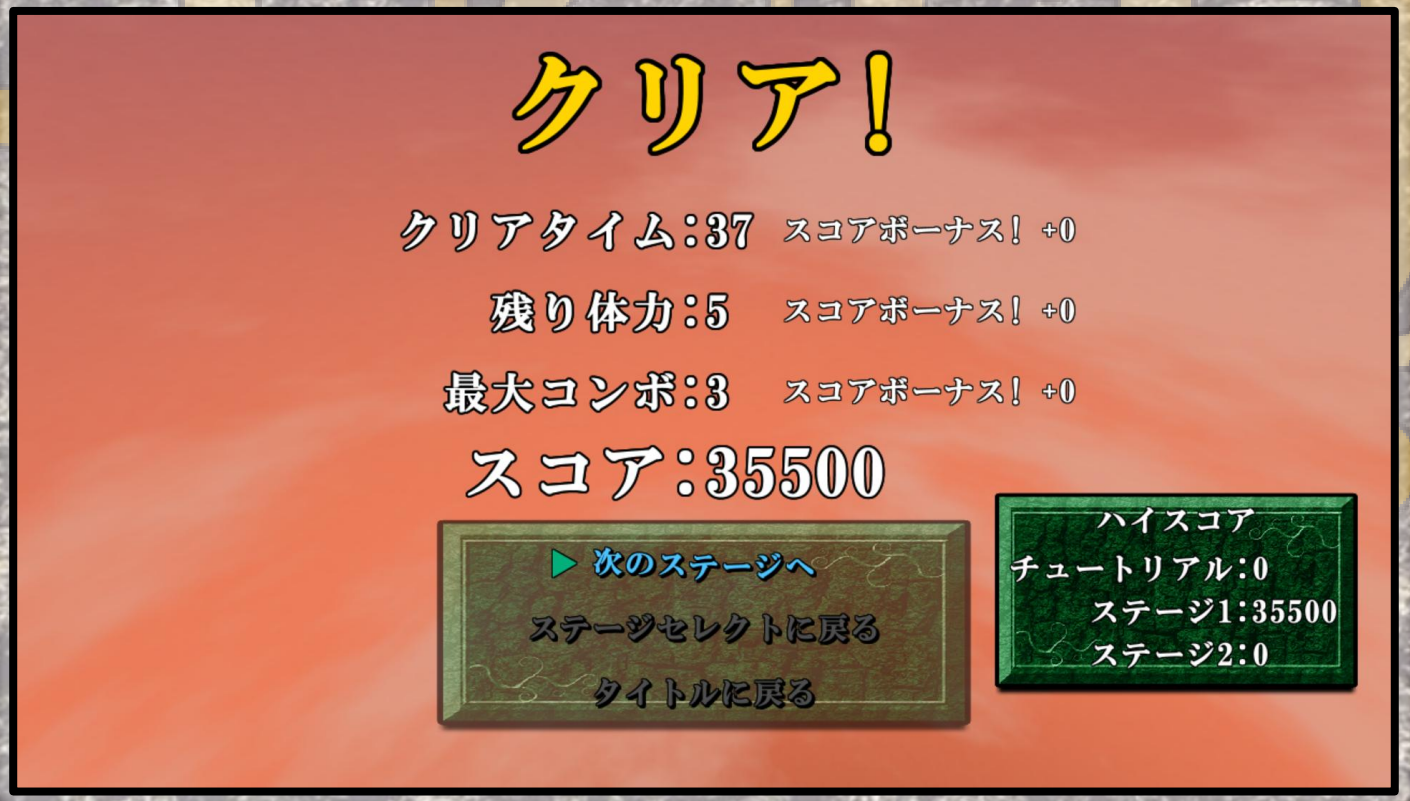
剣と盾を使って戦う3Dアクションゲーム





ゲーム概要

- すべての敵を倒すとゲームクリア
- 素早く効率的に敵を倒し、ハイスコアを目指せ！





こだわった点

こだわった点①

アクションのレスポンス



入力

入力に対して何も反応がないと、ユーザーとしては何が起こっているのか分からないと考えました。



入力

エフェクト、
SEなど

そこで、必ずエフェクトやSEなどの反応を入れることで、アクションを分かりやすくすることができました。

こだわった点①

アクションのレスポンス

実際のゲーム画面の例



こだわった点②

先行入力とキャンセル

攻撃の際に、あらかじめ入力があれば次の攻撃に移行できるようにすることで操作感の向上を図りました。

```cpp
void MyLib::PlayerController::UpdateInputBuffer()
{
    Input& input = Input::GetInstance();

    for (const auto& name : kBufferName)
    {
        // 押された瞬間から先行入力のフレームを設定する
        if (input.IsTriggered(name))
        {
            m_inputBufferFrames[name] = kInputBufferFrame;
        }
        // 先行入力している間はフレームを減少する
        else if (m_inputBufferFrames[name] > 0)
        {
            m_inputBufferFrames[name]--;
        }
    }
}
```

プレイヤーが入力のバッファを持つ

```cpp
bool MyLib::PlayerController::IsBuffered(const std::string& name) const
{
    // 現在の入力バッファに対象のキーが存在するかを確認
    auto it = m_inputBufferFrames.find(name);

    // 入力バッファにキーが存在し、先行入力と判定される場合trueを返す
    return it != m_inputBufferFrames.end() && it->second > 0;
}
```

この関数を呼ぶことで先行入力されているかを確認する

こだわった点②

先行入力とキャンセル

また、攻撃の途中で入力があった場合に攻撃をキャンセルして
別の行動にシームレスに移行できるようにこだわりました。



攻撃

先行入力



ローリング

or

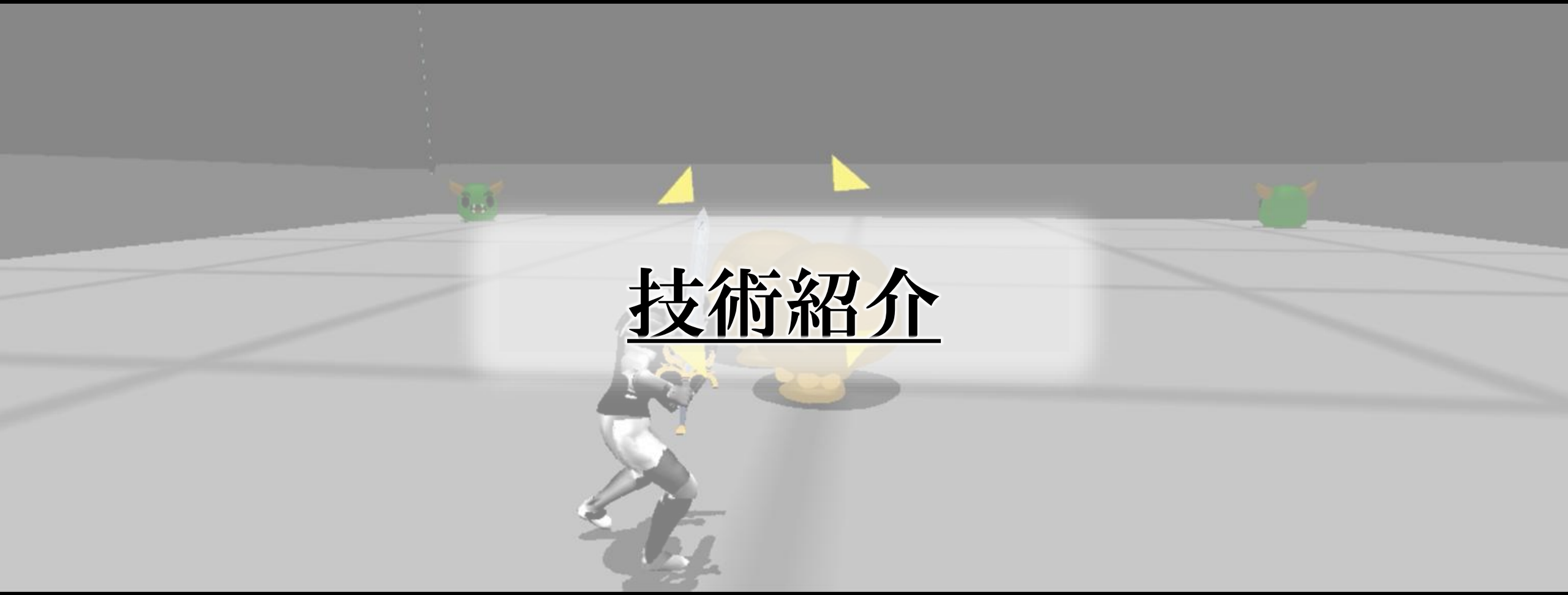


次の攻撃

or



ガード



技術紹介

技術紹介①

コンポーネント指向

以前の実装

3D描画

2D描画

当たり判定

以前の制作で、描画の処理が別々になっており、2Dと3Dの描画順の変更や
当たり判定の複数登録をするのに苦労したことがありました。

技術紹介①

コンポーネント指向

実際のコード

```
void Player::Init()
{
    AddComponent<MyLib::Animator>();                                // Animatorコンポーネントを追加
    AddComponent<MyLib::Rigidbody>();                               // Rigidbodyコンポーネントを追加
    AddComponent<MyLib::Collidable>();                              // Collidableコンポーネントを追加
    AddComponent<MyLib::Drawable3D>(MyLib::DrawLayer::Opaque);      // 3D描画コンポーネントを追加
    AddComponent<MyLib::PlayerController>();                        // PlayerControllerコンポーネントを追加
    AddComponent<MyLib::EffectComponent>();                         // エフェクトコンポーネントを追加

    // コンポーネントの初期化処理を行う
    GameObject::Init();
}
```

↑コンポーネントを追加する関数を呼ぶだけで機能が追加できます。

そこで、コンポーネント指向を用いることで、必要な処理のみを行うことができ、機能の追加も容易に行えるようになりました。

技術紹介②

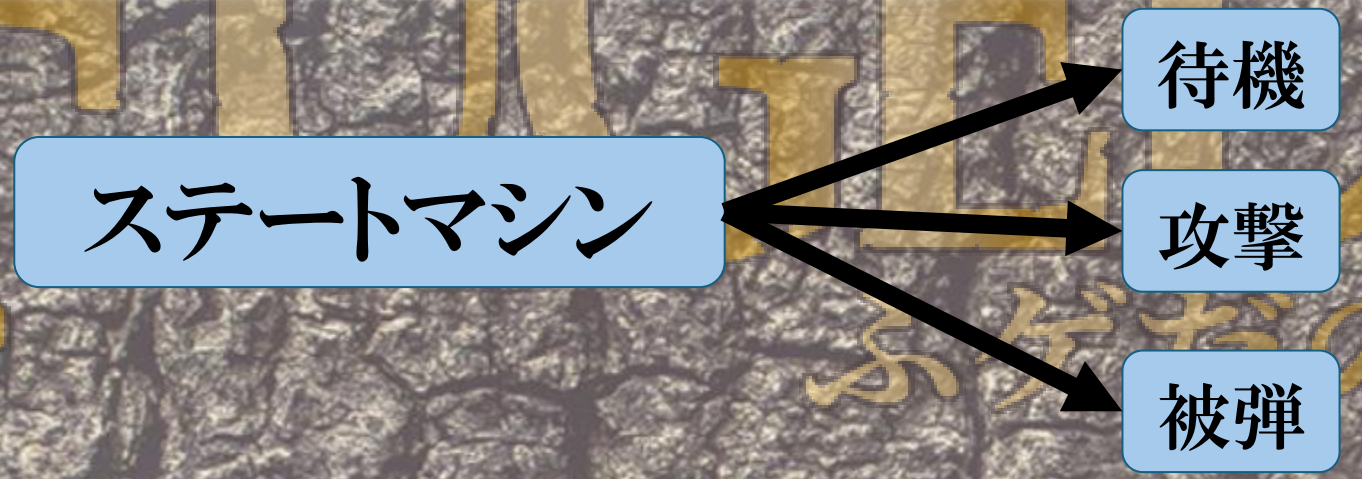
ステートマシン

ChangeStateを呼ぶだけで状態を変更することができる

実際のコード

```
void MyLib::PlayerController::Init(std::weak_ptr<MyLib::Game
{
    m_stateMachine.Init(this);

    // 最初の状態を待機状態にする
    m_stateMachine.ChangeState<MyLib::PlayerStateIdle>();
```

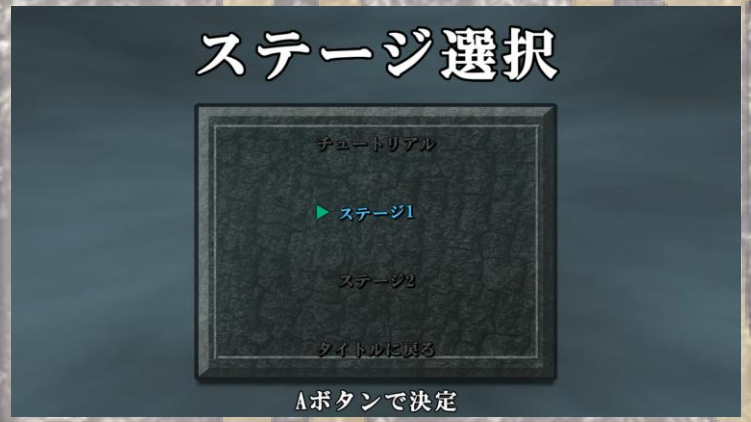


プレイヤーや敵の状態遷移にステートマシンを使用し、クラスごとの行数を
少なくすることで、コードの可読性やステートの遷移をわかりやすくしました。

技術紹介③

非同期ロード

ステージを選ぶ



非同期ロードを行う



ロードしたリソースでゲームを進める



非同期ロードを行い、必要なリソースのみを使うことで、
より効率よくメモリを管理することができました。